Praktikum 5
ETL dengan Menggunakan Apache Airflow

1. Buat environment baru dengan nama *airflow_env*

```
conda create --name airflow_env
```

2. Aktifkan environment *airflow_env*

```
conda activate airflow_env
```

Step 1 dan 2 bisa juga dilakukan dengan syntax ini

```
python3.10 -m venv airflow_env
source airflow_env/bin/activate
```

3. Upgrade pip

```
pip install --upgrade pip
```

4. Install Apache Airflow versi 2.9.3 dengan Python version 3.10

```
pip install "apache-airflow==2.9.3" \
--constraint
"https://raw.githubusercontent.com/apache/airflow/constraints-
2.9.3/constraints-3.10.txt"
```

Jika Apache-Airflow ini bermasalah ketika inisialisasi database, maka gunakan syntax ini

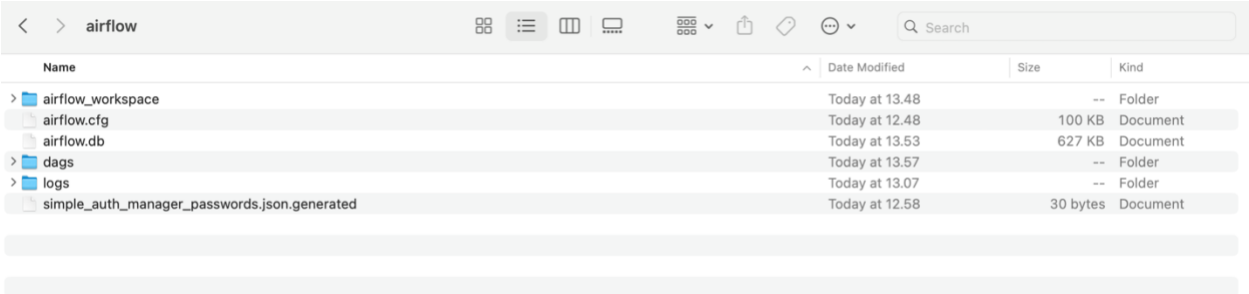
```
pip3 install 'apache-airflow[postgres,google,gcp,sentry,statsd]==3.1.3' \
 --constraint
"https://raw.githubusercontent.com/apache/airflow/constraints-
3.1.3/constraints-3.10.txt"
```

5. Cek direktori/folder instalasi Airflow. Kalau di Mac, biasanya otomatis file-file instalasi akan ditaruh di folder */Users/sitimariyah/airflow* dengan sitimariyah adalah nama user.

6. Buat direktori/folder *dags* di dalam folder *airflow*

```
mkdir dags
```

Kemudian, cek folder yang ada. Jika berhasil maka susunan folder akan seperti ini:



7. Lakukan inisialisasi database

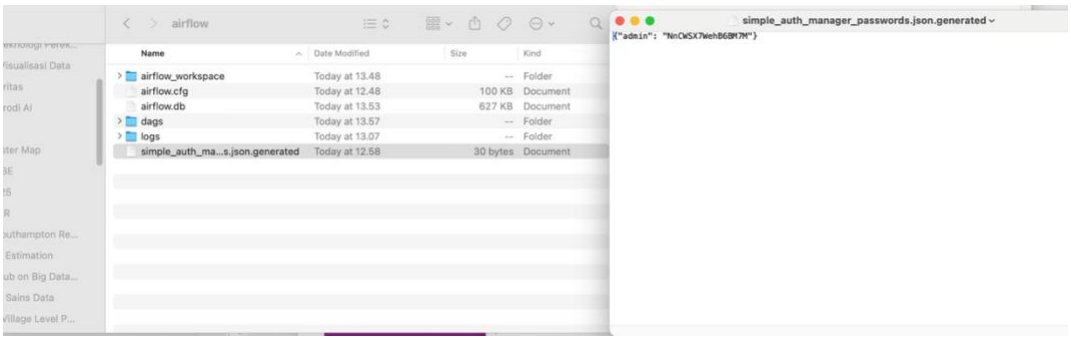
```
airflow db migrate
```

Jika terdapat eror maka sebelum menjalankan syntax ini, hapus terlebih dahulu file airflow.db pada direktori airflow.

8. Pada dasarnya, users untuk database sudah dibuat dengan username *admin* dan role *admin*. Ini bisa dicek di file airflow.cfg

Pada screenshot, kita sudah memiliki satu user dengan username *admin* dan role *admin*.

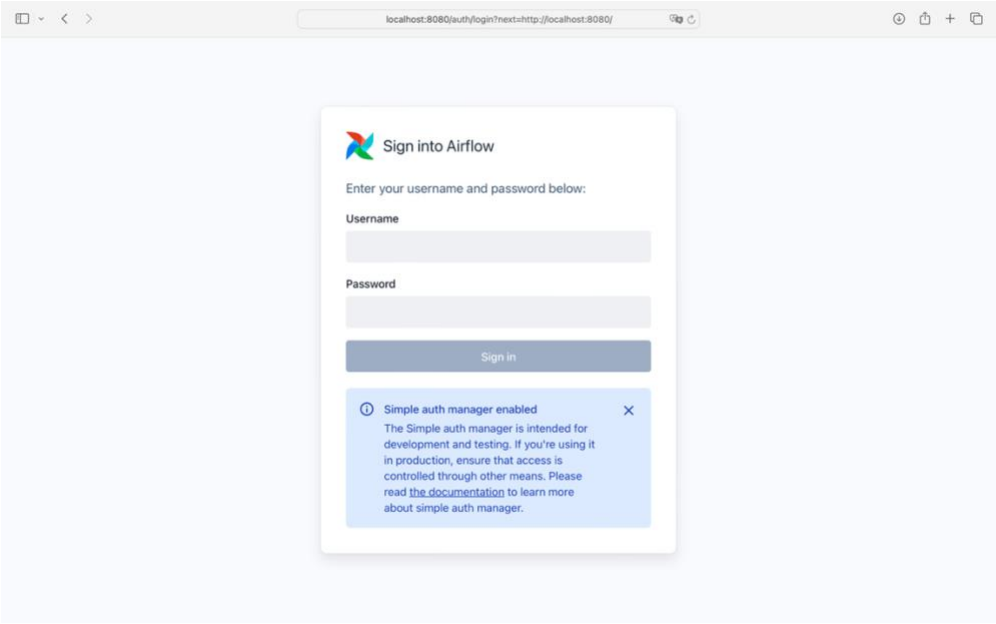
9. Untuk mengetahui password dari user *admin* tersebut, bisa dicek di file *simple_auth_manager_passwords.json.generated*. Jika dibuka file tersebut dengan text editor maka akan muncul password untuk user *admin*.



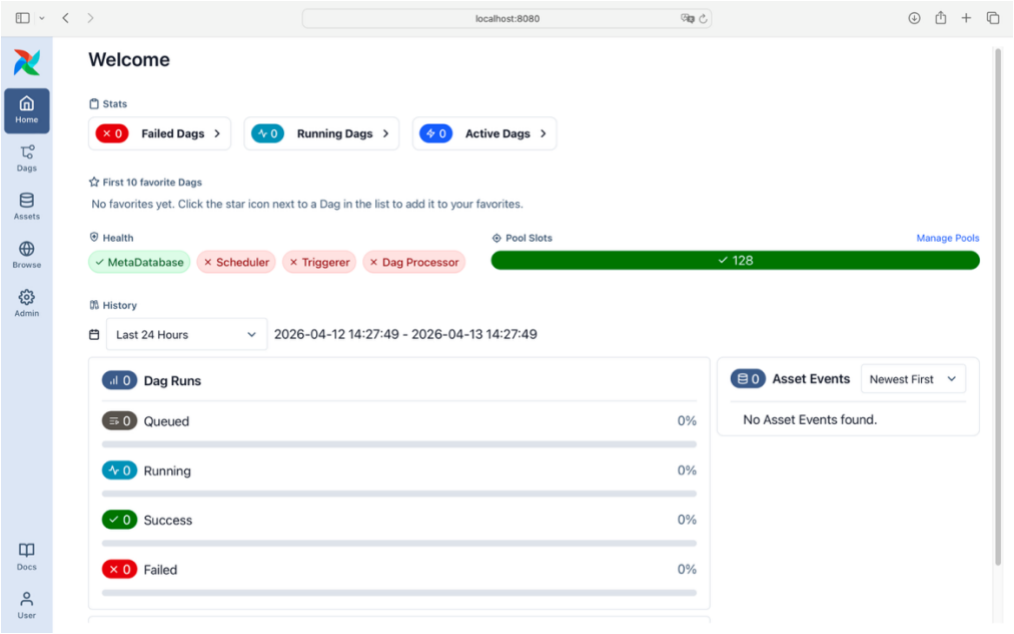
10. Jalankan airflow sehingga bisa kita akses melalui browser pada localhost

```
airflow api-server --port 8080
```

Buka browser dan akses alamat: localhost:8080



11. Masukkan username dan password untuk user admin untuk Sign in. Jika berhasil, maka akan muncul tampilan Home Airflow seperti ini:



Dari tampilan Home, terlihat bahwa belum ada DAG (*Directed Acyclic Graph*) yang gagal, sedang berjalan maupun yang aktif. Pada tutorial ini, kita akan membuat dan mengaplikasikan DAG.

12. Jalankan Airflow Scheduler

```
airflow scheduler
```

Output:

13. Jalankan Airflow-dag processor

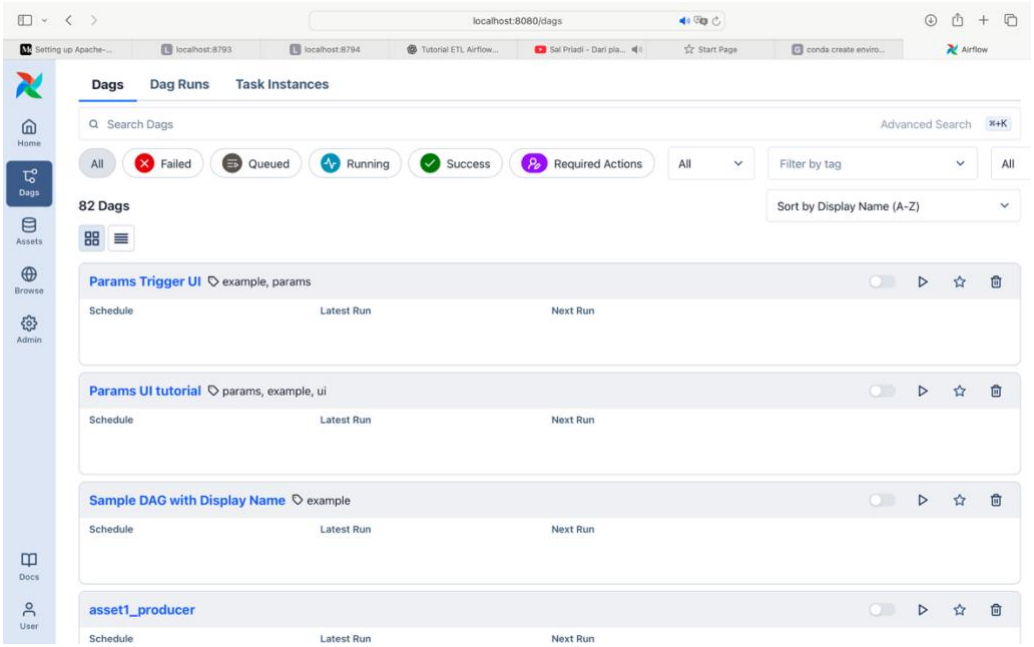
```
airflow dag-processor
```

Kode ini akan memproses semua dag dan menampilkannya dalam list dags di UI Airflow yang bisa dilihat di localhost:8080

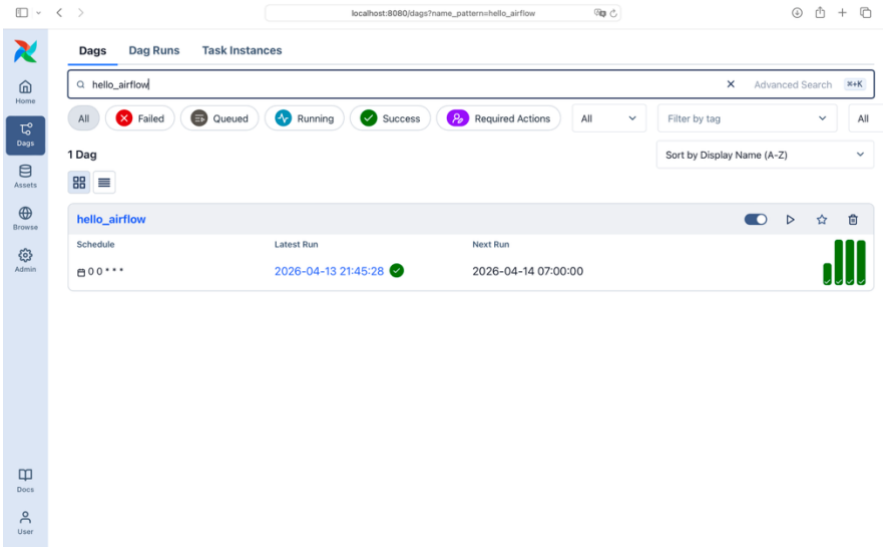
Output:

```
...pp/bin/airflow api-server --port 8080   ~/airflow/dags — -zsh   ...ashboard_app/bin/airflow scheduler   ...ard_app/bin/airflow dag-processor   ...ashboard_app/bin/airflow triggerer
(dashboard_app) sitimariyah@Sitis-MacBook-Pro-2 airflow % airflow dag-processor
2026-04-13T14:03:42.514901Z [info     ] NumExpr defaulting to 8 threads. [numexpr.utils] loc=utils.py:164
2026-04-13T14:03:42.775525Z [info     ] Starting the Dag Processor Job [airflow.jobs.dag_processor_job_runner.DagProcessorJobRunner] loc=dag_processor_job_runner.py:59
2026-04-13T14:03:42.775918Z [info     ] Processing files using up to 2 processes at a time  [airflow.dag_processing.manager.DagFileProcessorManager] loc=manager.py:265
2026-04-13T14:03:42.776977Z [info     ] Process each file at most once every 30 seconds [airflow.dag_processing.manager.DagFileProcessorManager] loc=manager.py:266
2026-04-13T14:03:42.777163Z [info     ] DAG bundles loaded: dags-folder, example_dags [airflow.dag_processing.bundles.manager.DagBundlesManager] loc=manager.py:179
2026-04-13T14:03:42.781956Z [info     ] DAG bundles loaded: dags-folder, example_dags [airflow.dag_processing.bundles.manager.DagBundlesManager] loc=manager.py:179
2026-04-13T14:03:42.782160Z [info     ] Checking for new files in bundle dags-folder every 300 seconds [airflow.dag_processing.manager.DagFileProcessorManager] loc=manager.py:276
2026-04-13T14:03:42.782211Z [info     ] Checking for new files in bundle example_dags every 300 seconds [airflow.dag_processing.manager.DagFileProcessorManager] loc=manager.py:276
2026-04-13T14:03:42.787015Z [info     ] Refreshing bundle dags-folder  [airflow.dag_processing.manager.DagFileProcessorManager] loc=manager.py:553
2026-04-13T14:03:42.788529Z [info     ] Searching for files in dags-folder at /Users/sitimariyah/airflow/dags [airflow.dag_processing.manager.DagFileProcessorManager] loc=manager.py:604
2026-04-13T14:03:42.790260Z [info     ] Found 2 files for bundle dags-folder [airflow.dag_processing.manager.DagFileProcessorManager] loc=manager.py:686
2026-04-13T14:03:42.802799Z [info     ] Refreshing bundle example_dags [airflow.dag_processing.manager.DagFileProcessorManager] loc=manager.py:553
2026-04-13T14:03:42.803799Z [info     ] Searching for files in example_dags at /Users/sitimariyah/conda/envs/dashboard_app/lib/python3.10/site-packages/airflow/example_dags [airflow.dag_processing.manager
.DagFileProcessorManager] loc=manager.py:604
2026-04-13T14:03:42.831813Z [info     ] Found 61 files for bundle example_dags [airflow.dag_processing.manager.DagFileProcessorManager] loc=manager.py:686
/Users/sitimariyah/conda/envs/dashboard_app/lib/python3.10/site-packages/airflow/api_fastapi/execution_api/routes/__init__.py:23 DeprecationWarning: 'HTTP_422_UNPROCESSABLE_ENTITY' is deprecated. Use 'HTT
P_422_UNPROCESSABLE_CONTENT' instead.
2026-04-13T14:03:43.168333Z [info     ]
================================================================================
DAG File Processing Stats

Bundle        File Path                                                  PID  Current Duration     # DAGs   # Errors  Last Duration    Last Run At
------------  ---------------------------------------------------------  -----  ------------------  --------  --------  ---------------  -------------
dags-folder   hello_airflow.py                                                                          0         0
dags-folder   hello_world.py                                                                            0         0
example_dags  tutorial.py                                                                               0         0
example_dags  example_trigger_target_dag.py                                                             0         0
example_dags  example_setup_teardown_taskflow.py                                                        0         0
example_dags  example_simplest_dag.py                                                                   0         0
example_dags  plugins/workday.py                                                                        0         0
example_dags  standard/example_bash_decorator.py                                                        0         0
example_dags  standard/example_trigger_controller_dag.py                 21827  0.04s                   0         0
example_dags  example_dynamic_task_mapping.py                                                           0         0
example_dags  example_workday_timetable.py                                                              0         0
example_dags  example_outlet_event_extra.py                                                             0         0
example_dags  example_custom_weight.py                                                                  0         0
example_dags  standard/example_latest_only.py                                                           0         0
example_dags  standard/example_python_decorator.py                                                      0         0
example_dags  example_dynamic_task_mapping_with_no_taskflow_operators.py                                0         0
example_dags  standard/example_branch_day_of_week_operator.py                                           0         0
example_dags  standard/example_branch_operator.py                                                       0         0
example_dags  example_xcomargs.py                                                                       0         0
example_dags  standard/example_external_task_child_deferrable.py                                        0         0
example_dags  example_asset_alias.py                                                                    0         0
example_dags  plugins/decreasing_priority_weight_strategy.py                                            0         0
example_dags  standard/example_short_circuit_operator.py                 21828  0.03s                   0         0
example_dags  standard/example_external_task_marker_dag.py                                              0         0
example_dags  standard/example_short_circuit_decorator.py                                               0         0
example_dags  example_inlet_event_extra.py                                                              0         0
example_dags  example_params_ui_tutorial.py                                                             0         0
example_dags  example_passing_params_via_test_command.py                                                0         0
example_dags  standard/example_sensor_decorator.py                                                      0         0
example_dags  standard/example_hitl_operator.py                                                         0         0
example_dags  standard/example_bash_operator.py                                                         0         0
example_dags  example_nested_branch_dag.py                                                              0         0
example_dags  example_time_delta_sensor_async.py                                                        0         0
example_dags  example_task_group.py                                                                     0         0
example_dags  example_complex.py                                                                        0         0
example_dags  example_local_kubernetes_executor.py                                                      0         0
example_dags  example_branch_python_dop_operator_3.py                                                   0         0
```

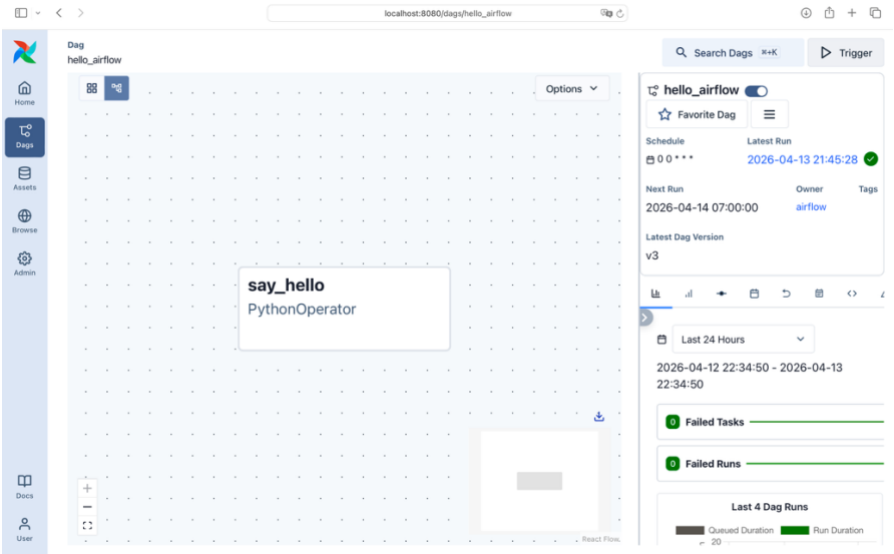
Seperti output di bawah ini ada 82 dags. Jumlah dags bisa berbeda antar versi Airflow.



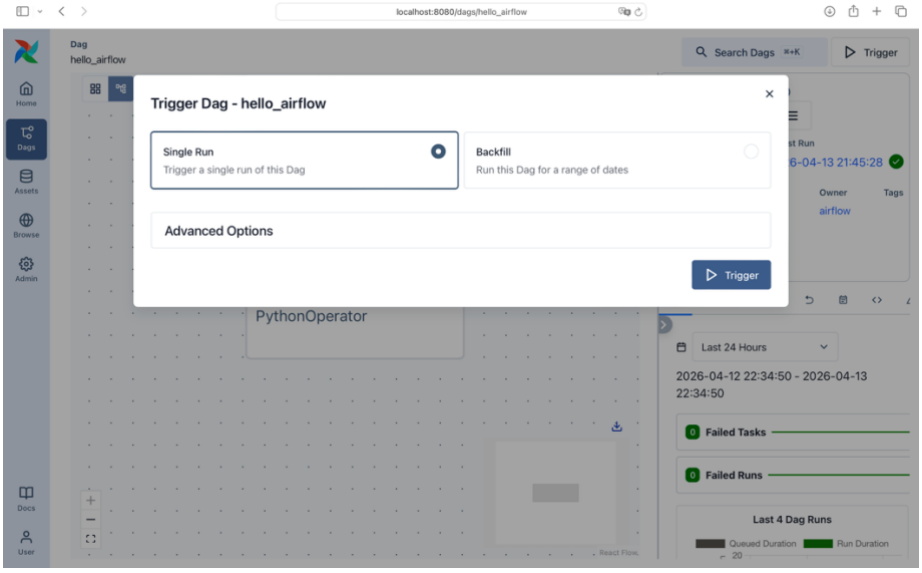
14. Lakukan tes apakah dag berjalan atau tidak dengan kode hello_airflow.py. Pada tab Dags, ketik hello_airflow maka akan keluar dag hello_airflow. Klik dag tersebut.



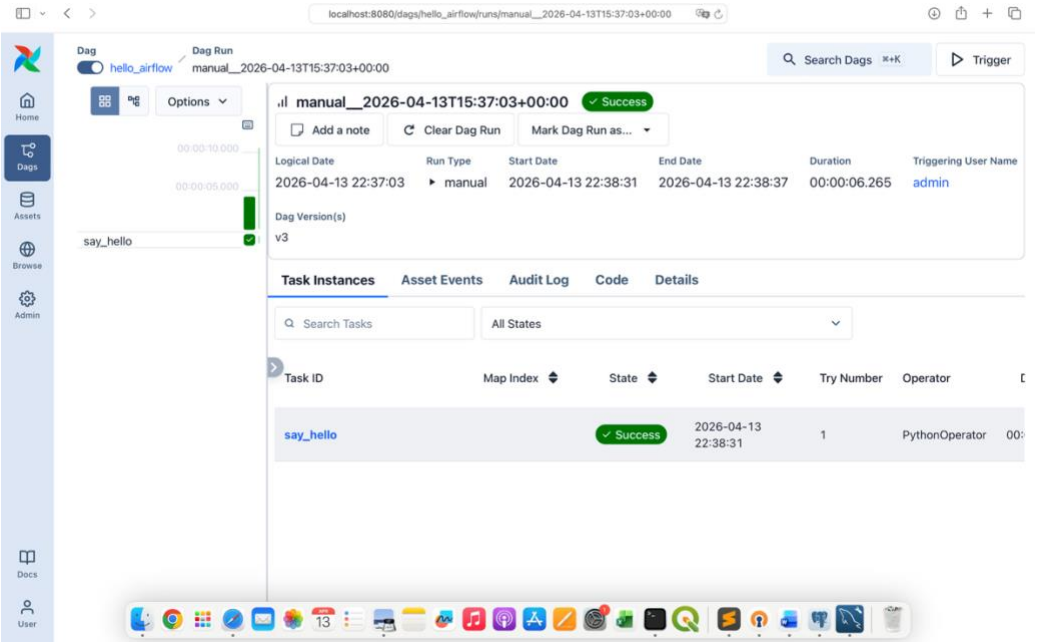
15. Untuk menjalankan dag hello_airflow, klik Trigger pada pojok kanan atas.



Setelah tombol Trigger diklik maka akan muncul sebuah pop-up windows, pilih Single Run lalu klik kembali tombol Trigger.



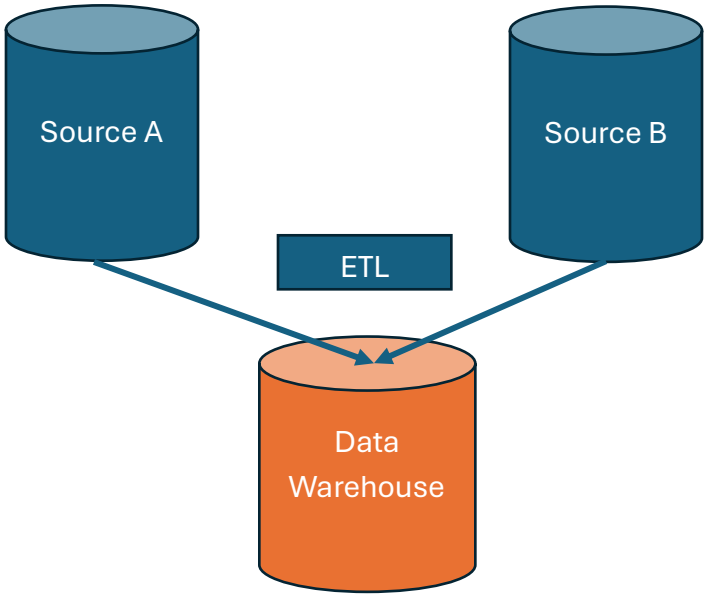
Output: jika berhasil dijalankan maka akan ada tulisan “Success”



Praktikum ETL dengan Airflow

Skema ETL kali ini sama dengan Praktikum Data Warehouse dan ETL menggunakan PySpark pada pertemuan 4. Kita akan membuat dua database transaksional sebagai dua data source. Source pertama (A) menggunakan PostgreSQL dan Source kedua menggunakan MySQL Workbench. Setiap Source menyimpan data operasional mentah (OLTP) dengan struktur:

1. Tabel fact_sales menyimpan transaksi penjualan
2. Tabel dim_product menyimpan data produk
3. Tabel dim_store menyimpan data toko/cabang
4. Tabel dim_date menyimpan referensi waktu.



Catatan: gunakan kode SQL yang sama seperti pada Modul 4 untuk membuat database source_a, source_b, dan database dwh.

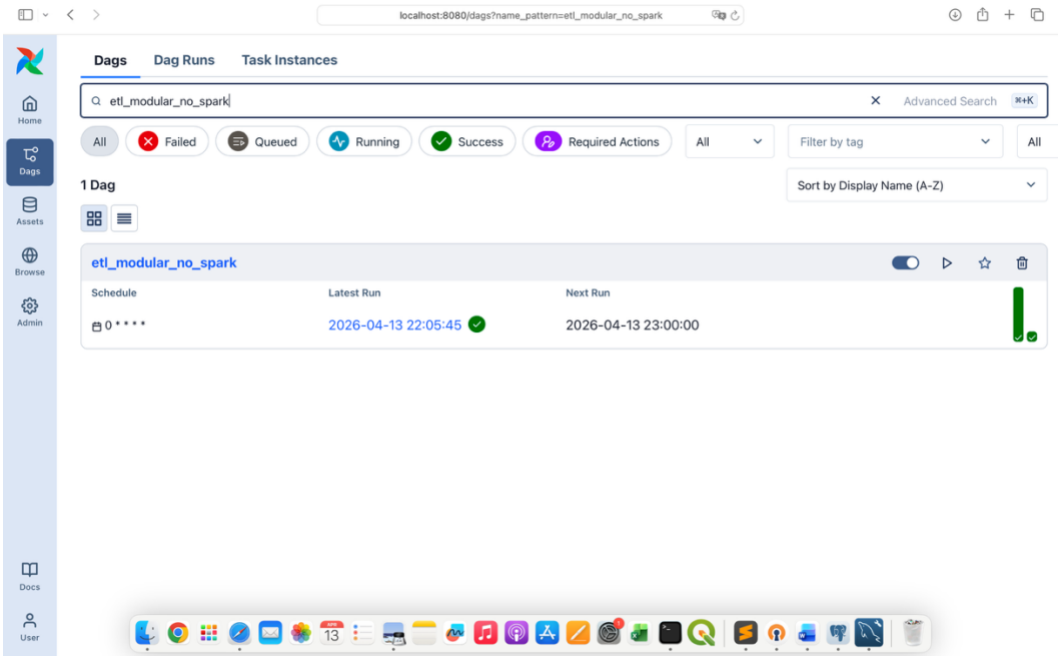
Kode Python untuk melakukan ETL dengan Airflow, simpan kode ini dengan nama file etl_airflow_modular.py

```
from airflow import DAG
from airflow.operators.python import PythonOperator
from datetime import datetime, timedelta
import pandas as pd
from sqlalchemy import create_engine

# ==============================
# KONFIGURASI DATABASE
# ==============================
# Menggunakan kredensial dari file dw_etl_final.py
SRC_A_URL = "mysql+pymysql://root:admin123@localhost:3306/source_a"
SRC_B_URL = "postgresql+psycopg2://postgres:admin@localhost:5432/source_b"
DWH_URL   = "mysql+pymysql://root:admin123@localhost:3306/dwh"

default_args = {
    'owner': 'airflow',
    'start_date': datetime(2026, 4, 1),
    'retries': 1,
    'retry_delay': timedelta(minutes=5)
}

# ==============================
# FUNGSI ETL PER TASK
# ==============================

def extract_source_a(**kwargs):
    """Task 1: Ekstraksi data dari MySQL Source A"""
    engine = create_engine(SRC_A_URL)
    tables = ["dim_product", "dim_store", "dim_date", "fact_sales"]
    data = {t: pd.read_sql(f"SELECT * FROM {t}", engine) for t in tables}

    # Simpan ke XCom agar bisa dibaca task selanjutnya (sebagai dictionary)
    # Catatan: Untuk data besar, sebaiknya simpan ke CSV sementara/S3/GCS
    return {t: df.to_json() for t, df in data.items()}

def extract_source_b(**kwargs):
    """Task 2: Ekstraksi data dari PostgreSQL Source B"""
    engine = create_engine(SRC_B_URL)
    tables = ["dim_product", "dim_store", "dim_date", "fact_sales"]
    data = {t: pd.read_sql(f"SELECT * FROM {t}", engine) for t in tables}
    return {t: df.to_json() for t, df in data.items()}

def transform_data(**kwargs):
    """Task 3: Transformasi, Union, dan Surrogate Key"""
    ti = kwargs['ti']
    # Ambil data dari task sebelumnya via XCom
    json_a = ti.xcom_pull(task_ids='extract_mysql_a')
    json_b = ti.xcom_pull(task_ids='extract_postgres_b')

    dfs_a = {t: pd.read_json(js) for t, js in json_a.items()}
    dfs_b = {t: pd.read_json(js) for t, js in json_b.items()}

    # Tambahkan Source System
```

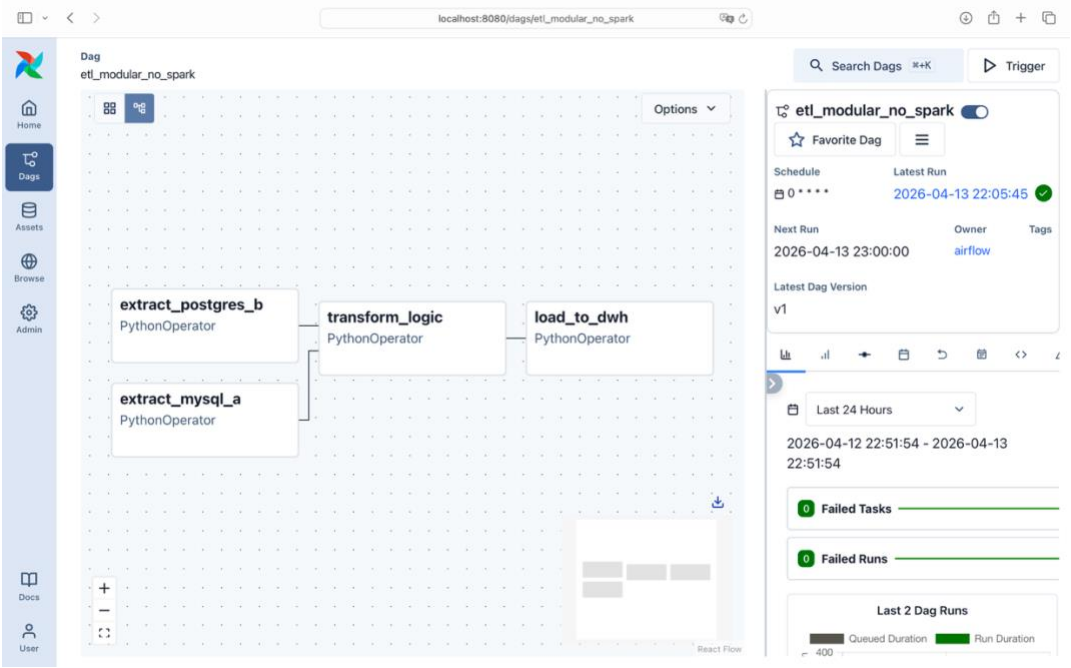
```
    for t in dfs_a: dfs_a[t]['source_system'] = 'A'
    for t in dfs_b: dfs_b[t]['source_system'] = 'B'

    # Union Data
    df_product = pd.concat([dfs_a["dim_product"], dfs_b["dim_product"]],
ignore_index=True)
    df_store   = pd.concat([dfs_a["dim_store"], dfs_b["dim_store"]],
ignore_index=True)
    df_sales   = pd.concat([dfs_a["fact_sales"], dfs_b["fact_sales"]],
ignore_index=True)
    df_date    = dfs_a["dim_date"]

    # Standardisasi Kategori
    cat_map = {'Makanan': 'Food', 'Food': 'Food', 'Minuman': 'Beverage',
'Drink': 'Beverage'}
    df_product['category'] =
df_product['category'].map(cat_map).fillna(df_product['category'])

    # Drop Duplicates
    df_product = df_product.drop_duplicates(['product_id', 'source_system'])
    df_store   = df_store.drop_duplicates(['store_id', 'source_system'])

    # Surrogate Key (Simulasi row_number)
    df_product['product_key'] = range(1, len(df_product) + 1)
    df_store['store_key'] = range(1, len(df_store) + 1)

    # Fact Enrichment (Mapping ID ke Key)
    df_fact = df_sales.merge(df_product[['product_id', 'source_system',
'product_key']], on=['product_id', 'source_system'], how='left')
    df_fact = df_fact.merge(df_store[['store_id', 'source_system',
'store_key']], on=['store_id', 'source_system'], how='left')

    df_fact_final = df_fact[['product_key', 'store_key', 'date_id',
'quantity', 'revenue', 'source_system']]
    df_fact_final = df_fact_final.rename(columns={'date_id': 'date_key'})

    return {
        "dim_product": df_product.to_json(),
        "dim_store": df_store.to_json(),
        "dim_date": df_date.to_json(),
        "fact_sales": df_fact_final.to_json()
    }

def load_to_dwh(**kwargs):
    """Task 4: Load hasil akhir ke MySQL DWH"""
    ti = kwargs['ti']
    final_data_json = ti.xcom_pull(task_ids='transform_logic')
    engine_dwh = create_engine(DWH_URL)

    for table_name, json_str in final_data_json.items():
        df = pd.read_json(json_str)
        # Load ke database (Mode overwrite/replace)
        df.to_sql(table_name, engine_dwh, if_exists='replace', index=False)

    print("Load ke DWH Selesai!")

# ==============================
```

```
# DEFINISI DAG & WORKFLOW
# ==============================
with DAG(
    'etl_modular_no_spark',
    default_args=default_args,
    schedule='@hourly',
    catchup=False
) as dag:

    t1 = PythonOperator(task_id='extract_mysql_a',
python_callable=extract_source_a)
    t2 = PythonOperator(task_id='extract_postgres_b',
python_callable=extract_source_b)
    t3 = PythonOperator(task_id='transform_logic',
python_callable=transform_data)
    t4 = PythonOperator(task_id='load_to_dwh', python_callable=load_to_dwh)

    # Visualisasi DAG: T1 & T2 jalan paralel, lalu T3, lalu T4
    [t1, t2] >> t3 >> t4
```

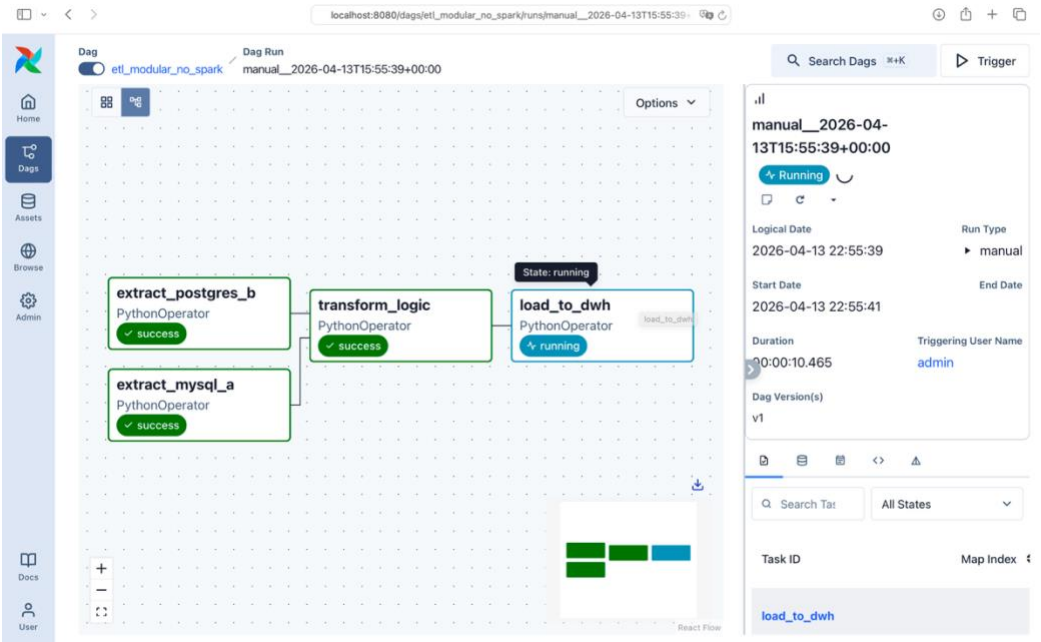
Shutdown Airflow server termasuk airflow scheduler dan airflow dag-processor, dengan menekan Ctrl+C. Lalu jalankan kembali `airflow scheduler`, `airflow dag-procesor`, dan `airflow api-server --port 8080`. Buka kembali localhost:8080 di browser dan search di Tab Dags `etl_modular_no_spark`.



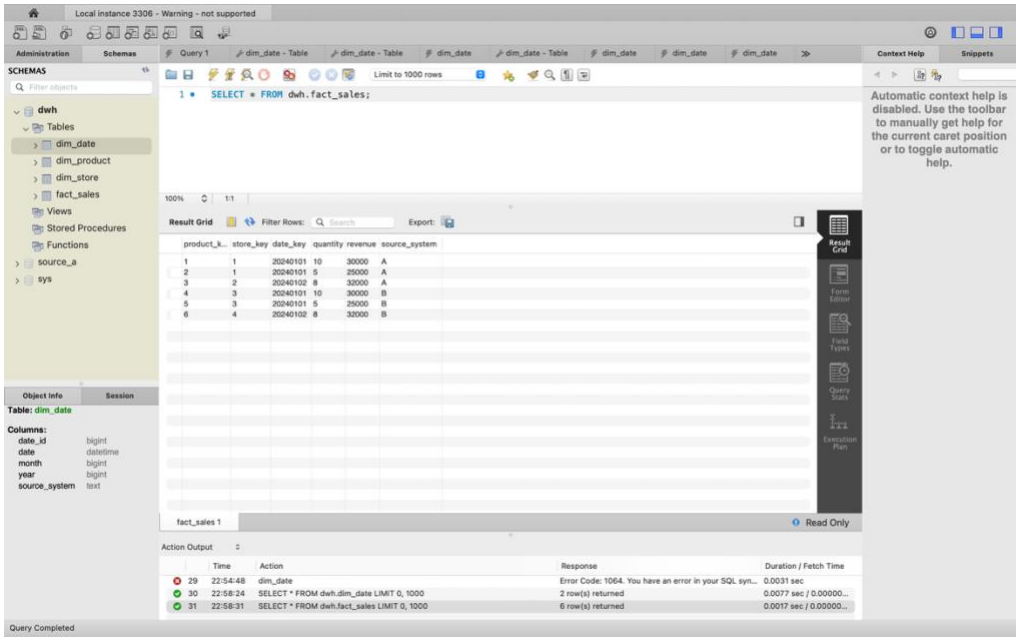
Klik dag `etl_modular_no_spark` tersebut. Klik Show Graph untuk menampilkan proses DAG (*Directed Acyclic Graph*) dari proses extract data dari dua database, melakukan transformasi, hingga loading data hasil transformasi ke data warehouse.



Klik Trigger pada pojok kanan atas. Sebelumnya, jangan lupa membuat database dwh di MySQL Workbench menggunakan sintaks SQL pada Modul 4. Proses melakukan ETL, jika berhasil maka status akan menjadi “success”, jika masih dijalankan akan berstatus “running”.



Jika running selesai dilakukan maka cek data warehouse dwh pada MySQL Workbench. Jika data warehouse dwh memiliki data maka proses ETL dengan Airflow berhasil. Contoh, lakukan browse data pada tabel fact_sales, jika datanya ada maka ETL dengan Airflow berhasil.



Referensi:

a. https://airflow.apache.org
b. https://medium.com/accredian/setting-up-apache-airflow-in-macos-57ab9e8edb8a